

MODULE 3

- Agile Development
- Principles that guide Practice

The Manifesto for Agile Software Development

“We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- *Individuals and interactions over processes and tools*
- *Working software over comprehensive documentation*
- *Customer collaboration over contract negotiation*
- *Responding to change over following a plan*

That is, while there is value in the items on the right, we value the items on the left more.”

Kent Beck et al

- **Agile methods were developed** in an effort to overcome perceived and actual weaknesses in conventional software engineering.
- Agile development can **provide important benefits**, but it is not applicable to all projects, all products, all people, and all situations.
- It is also **not antithetical** to solid software engineering practice and can be applied as an overriding philosophy for all software work.
- In the **modern economy**, it is often difficult or impossible to predict how a computer-based system (e.g., a Web-based application) will evolve as time passes.
- One of the most compelling characteristics of the agile approach is its ability to **reduce the costs of change** throughout the software process.
- Does this mean that a recognition of challenges posed by modern realities causes you to **discard valuable software engineering** principles, concepts, methods, and tools? Absolutely not!
- Alistair Cockburn [Coc02] argues that the prescriptive process models introduced in Chapter 2 have a major failing: *they forget the frailties of the people who build computer software*.
- If process models are to work, they must provide a **realistic mechanism** for encouraging the discipline that is necessary, or they must be characterized in a manner that shows “tolerance” for the people who do software engineering work.

What is AGILITY?

Ivar Jacobson [Jac02a] provides a useful discussion:

- *Agility has become today's buzzword when describing a modern software process.*
- *Everyone is agile.*
- An agile team is a nimble team able to **appropriately respond to changes**.
- **Change is what software development is very much about.** Changes in the software being built, changes to the team members, changes because of new technology, changes of all kinds that may have an impact on the product they build or the project that creates the product.
- **Support for changes** should be built-in everything we do in software, something we embrace because it is the heart and soul of software.
- An agile team recognizes that **software is developed by individuals working in teams** and that the skills of these people, their ability to collaborate is at the core for the success of the project.

What is “Agility”?

- In Jacobson’s view, the pervasiveness of change is the primary driver for agility.
- But agility is more than an effective response to change. It also encompasses the philosophy espoused in the manifesto noted at the beginning
- Effective (rapid and adaptive) **response to change** (team members, new technology, requirements)
- Encourages **team structure and attitude** that make communication easy
- Drawing the **customer into the team**. Eliminate “us and them” attitude.
- Planning in an uncertain world has its limits and plan must be **flexible**.
- Agility can be applied to any software process.
- Eliminate all but the most essential work products and keep them **lean**.
- Emphasize an **incremental** delivery strategy as opposed to intermediate products that gets working software to the customer as rapidly as feasible.

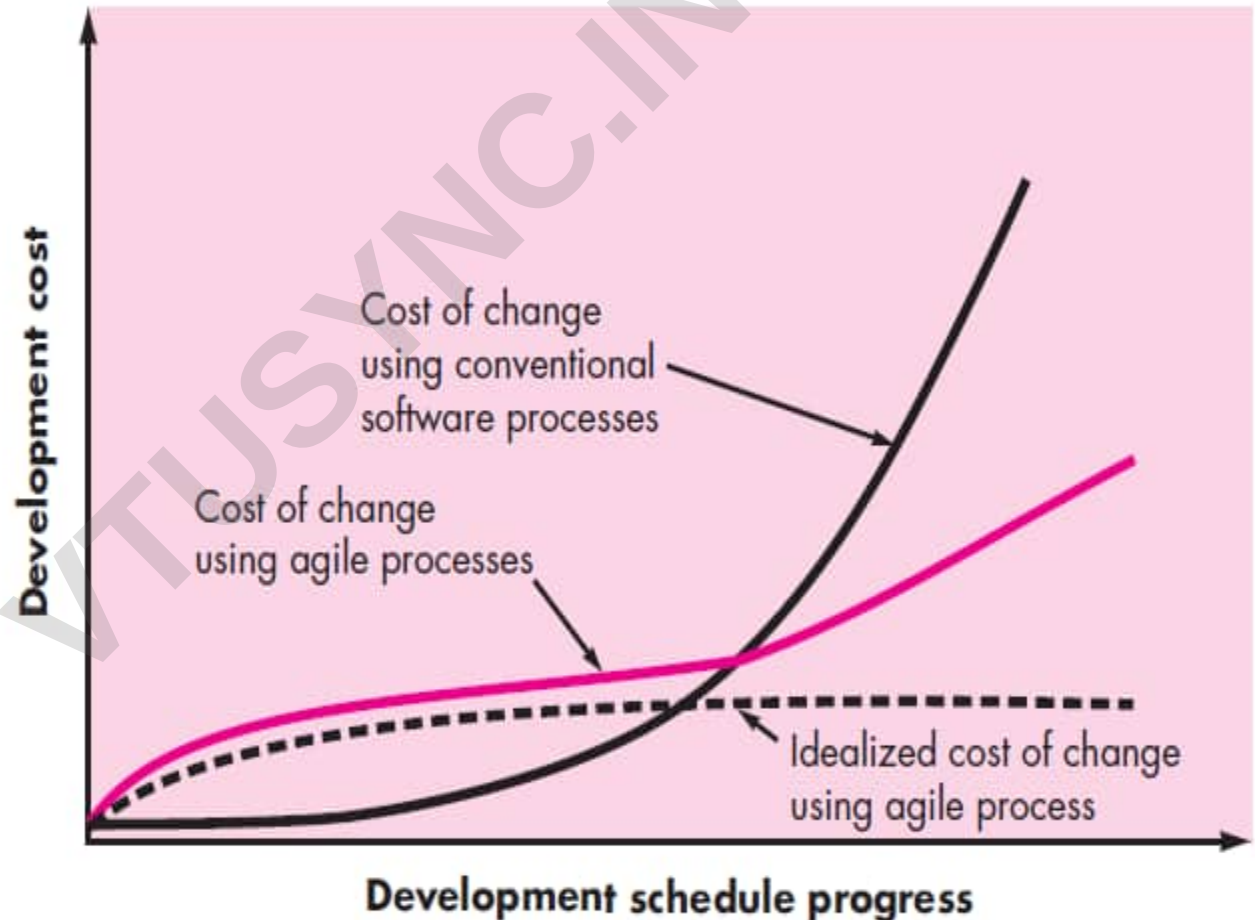
Agility and the Cost of Change

- **Conventional wisdom** is that the cost of change increases nonlinearly as a project progresses. It is relatively easy to accommodate a change when a team is gathering requirements early in a project. If there are any changes, the costs of doing this work are minimal. But if the middle of validation testing, a stakeholder is requesting a major functional change. Then the change requires a modification to the *architectural design, construction of new components, changes to other existing components, new testing and so on. Costs escalate quickly.*
- A well-designed **agile process** may “flatten” the cost of change curve by coupling **incremental delivery** with agile practices such as **continuous unit testing** and **pair programming**. Thus team can accommodate changes late in the software project without dramatic cost and time impact.

Agility and the Cost of Change

FIGURE 3.1

Change costs
as a function
of time in
development



note:

"Agility is dynamic,
content specific,
nonraccially"

What is An Agile Process

- An agile Process Is driven by **customer descriptions** of what is required (scenarios).
 - Some key assumptions about majority software projects:
 - Recognizes that plans are **short-lived** (some requirements will persist, some will change. Customer priorities will change)
 - **Design and construction are interleaved**, hard to say how much design is necessary before construction. Design models are proven as they are created.
 - Analysis, design, construction and testing are not predictable as we like
- Thus Agile Process has to **Adapt** as changes occur due to **unpredictability**
- Delivers multiple ‘software **increments**’, deliver an operational prototype or portion of an OS to collect customer feedback for adaption.

Agility Principles - I

The Agile Alliance (see [Agi03], [Fow01]) defines 12 agility principles for those who want to achieve agility:

1. Our highest priority is to **satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing** requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together **daily** throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face** conversation.

Agility Principles - II

7. **Working software** is the primary measure of progress.
8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain **a constant pace** indefinitely.
9. Continuous attention to **technical excellence** and **good design** enhances agility.
10. **Simplicity** – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing** teams.
12. At regular intervals, the team reflects on how to become more effective, then **tunes and adjusts** its behavior accordingly.

the principles define an *agile spirit that is maintained in* each of the process models presented in this chapter.

The Politics of Agile Development

- Jim Highsmith states the extremes when he characterizes the feeling of the pro-agility camp (“agilists”).

“Traditional methodologists are a bunch of stick-in-the-muds who’d rather produce flawless documentation than a working system that meets business needs.”

As a counterpoint, he states (again, facetiously) the position of the traditional software engineering camp:

“Lightweight, er, ‘agile’ methodologists are a bunch of glorified hackers who are going to be in for a heck of a surprise when they try to scale up their toys into enterprise-wide software.”

The Politics of Agile Development

- If warfare breaks out, rational thought disappears and beliefs rather than facts guide decision making.
- No one is against agility. The real question is: What is the best way to achieve it?
- Even within the agile school itself, there are many proposed process models (Section 3.4), each with a subtly different approach to the agility problem
- there is much that can be gained by considering the best of both schools and virtually nothing to be gained by denigrating either approach.

Human Factors

As Cockburn and Highsmith [Coc01a] state, “Agile development focuses on the talents and skills of individuals, molding the process to specific people and teams.”

The key point in this statement is that *the process molds to the needs of the people and team, not the other way around.*

- key traits must exist among the people on an agile team and the team itself:
 - **Competence.** (talent, skills, knowledge)
 - **Common focus.** (deliver a working software increment)
 - **Collaboration.** (peers and stakeholders)
 - **Decision-making ability.** (freedom to control its own destiny)
 - **Fuzzy problem-solving ability.**(ambiguity and constant changes, today problem may not be tomorrow's problem)
 - **Mutual trust and respect.** (jelled team)
 - **Self-organization.** (themselves for the work to be done, process to accommodate its local environment, the work schedule)

3.4 EXTREME PROGRAMMING (XP)

- *Extreme Programming (XP)*, the most widely used approach to agile software development.
- Although early work on the ideas and methods associated with XP occurred during the late 1980s, the seminal work on the subject has been written by Kent Beck [Bec04a].
- More recently, a variant of XP, called *Industrial XP (IXP)* has been proposed [Ker05].
- IXP refines XP and targets the agile process specifically for use within large organizations.

3.4.1 XP Values

•Beck [Bec04a] defines a set of five *values that establish a foundation for all work performed* as part of XP—**communication, simplicity, feedback, courage, and respect**. Each of these values is used as a driver for specific XP activities, actions, and tasks.

1. Communication

•In order to achieve effective *communication between software engineers and other stakeholders* XP emphasizes

- i. close, yet informal (verbal) collaboration between customers and developers,
- ii. the establishment of effective metaphors for communicating important concepts
- iii. continuous feedback
- iv. and the avoidance of voluminous documentation as a communication medium.

2. Simplicity

•To achieve *simplicity, XP restricts developers to design only for immediate needs*, rather than consider future needs.

- The intent is to create a simple design that can be easily implemented in code).
- If the design must be improved, it can be *refactored at* a later time.

3.4.1 XP Values

3. Feedback

- *Feedback is derived from three sources:*
 - the implemented software itself:*
 - the customer:*
 - and other software team members:*

4. Courage

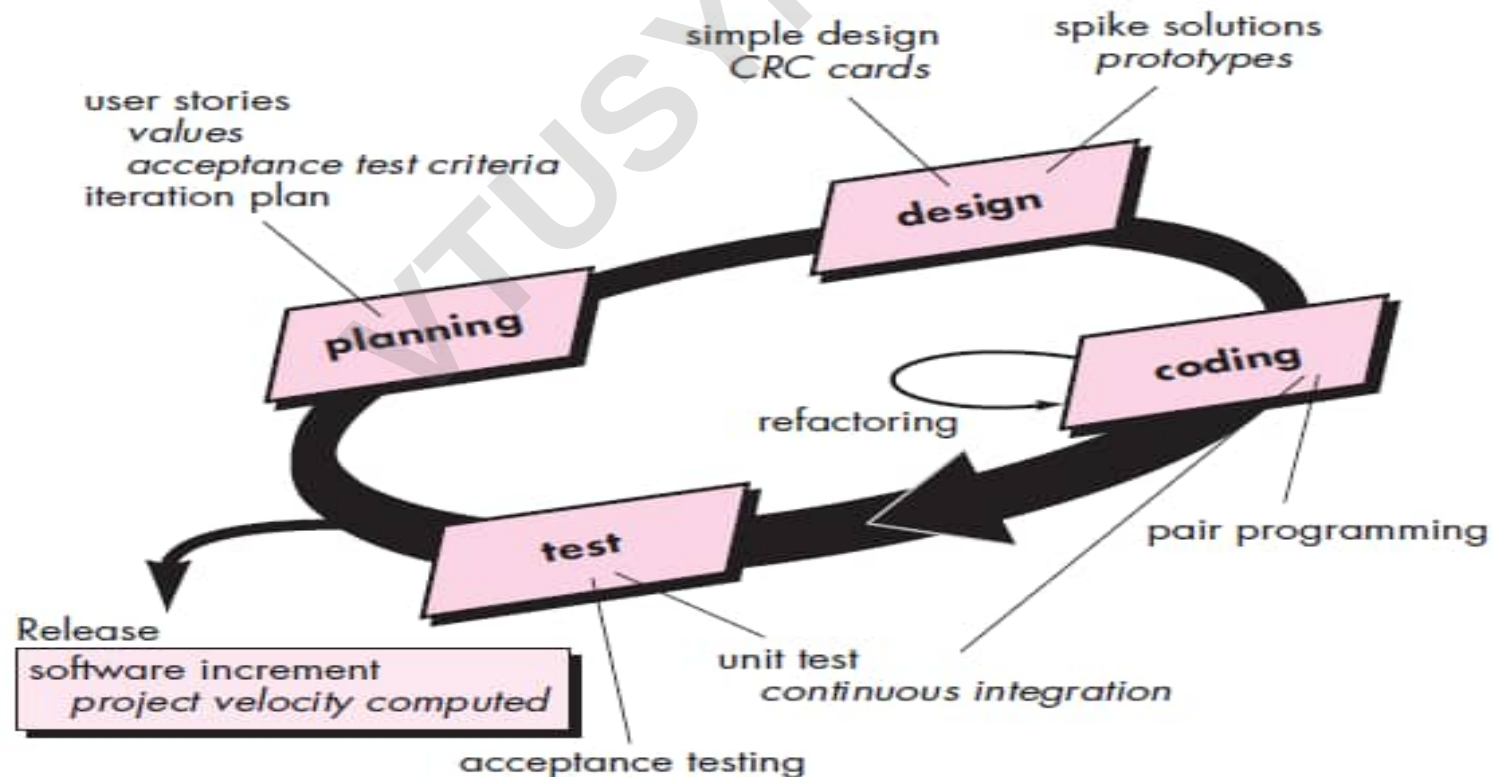
- Beck [Bec04a] argues that strict adherence to certain XP practices demands *courage*. *A better word might be discipline.*
- An agile XP team must have the discipline (courage) to design for today, recognizing that future requirements may change dramatically, thereby demanding substantial rework of the design and implemented code.

5. Respect

- By following each of these values, the agile team inculcates *respect among it* members, between other stakeholders and team members, and indirectly, for the software itself.
- As they achieve successful delivery of software increments, the team develops growing respect for the XP process.

3.4.2 The(XP) Process

- Extreme Programming uses an object-oriented approach as its preferred development paradigm and encompasses a set of rules and practices that occur within the context of four framework activities: **planning, design, coding, and testing.**
 - Figure 3.2 illustrates the XP process and notes some of the key ideas and tasks that are associated with each framework activity.
-



1. XP Planning

- Begins with the listening, leads to creation of “**user stories**” that describes required output, features, and functionality. Customer assigns a value(i.e., a priority) to each story.
- Agile team assesses each story and assigns a **cost** (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
- Working together, stories are grouped for a **deliverable increment next release**.
- A **commitment** (stories to be included, delivery date and other project matters) is made. Three ways: 1. Either all stories will be implemented in a few weeks, 2. high priority stories first, or 3. the riskiest stories will be implemented first.
- After the first increment “**project velocity**”, namely number of stories implemented during the first release is used to 1. help define subsequent delivery dates for other increments. 2. determine whether an over commitment has been made for all stories
- **Customers can** add stories, delete existing stories, change values of an existing story, split stories as development work proceeds.

2. XP Design

Follows the **KIS principle (keep it simple)**. A simple design is always preferred over a more complex representation.

- The design provides **implementation guidance** for a story as it is written—nothing less, nothing more.
- Encourage the use of **CRC (class-responsibility-collaborator) cards** in an object-oriented context. The only design work product of XP. They identify and organize the classes that are relevant to the current software increment.
- For difficult design problems, XP suggests the immediate creation of an operational prototype of that portion of the design. Called a *spike solution*. “**spike solutions**”—a design prototype is implemented and evaluated. The intent is to lower risk when true implementation starts and to validate the original estimates for the story containing the design problem.
- Encourages “**refactoring**”—an iterative refinement of the internal program design. Does not alter the external behavior yet improve the internal structure. Minimize chances of bugs. More efficient, easy to read.
- A **central notion** in XP is that design occurs both before *and after coding commences*.

3. XP Coding

- Recommends the **construction of a unit test** for a story *before* coding commences. So implementer can focus on what must be implemented to pass the test.
- Once the code is complete, it can be unit-tested immediately, thereby providing instantaneous feedback to the developers.
- Encourages “**pair programming**”. Two people work together at one workstation. This provides a mechanism for Real time problem solving, real time quality assurance. Each person Take slightly different roles.
- As pair programmers complete their work, the code they **develop is integrated with the work of others**. This “continuous integration” strategy helps to avoid compatibility and interfacing problems and provides a “**smoke testing**” environment that helps to uncover errors early.

4. XP Testing

- the creation of unit tests before coding commences is a key element of the XP approach
 - The **unit test should be automated**. Regression tests are conducted whenever codes are modified
 - **Integration and Validation testing** of the system can occur on daily basis. This provides the XP team with a continual indication of progress and also can raise warning flags early if things go awry.
 - **“XP Acceptance tests”** are defined by the customer and executed to assess customer visible functionality

3.4.3 Industrial XP

•Joshua Kerievsky [Ker05] describes *Industrial Extreme Programming (IXP)* in the following manner: “***IXP is an organic evolution of XP. It is imbued with XP’s minimalist ,customer-centric, test-driven spirit. IXP differs most from the original XP in its greater inclusion of management, its expanded role for customers, and its upgraded technical practices.***”

•IXP incorporates six new practices that are designed to help ensure that an XP project works successfully for significant projects within a large organization.

1. Readiness assessment
2. Project Community
3. Project Chartering
4. Test Driven Management
5. Retrospectives
6. Continuous learning

3.4.3 Industrial XP (Conti..)

1. Readiness assessment:

Prior to the initiation of an IXP project, the organization should conduct a *readiness assessment*. The assessment ascertains whether

- (1) an appropriate **development environment** exists to support IXP,
- (2) the team will be populated by the proper set of **stakeholders**,
- (3) the organization has a distinct **quality program** and supports continuous improvement,
- (4) the **organizational culture** will support the new values of an agile team, and
- (5) the **broader project community** will be populated appropriately.

2. Project community:

- **Classic XP suggests that the right people be used to** populate the agile team to ensure success.
- The implication is that people on the team must be well-trained, adaptable and skilled, and have the proper temperament to contribute to a self-organizing team.
- When XP is to be applied for a significant project in a large organization, the concept of the “team” should morph into that of a *community*.
- *A community may have a technologist and customers who are central to the success of a project as well as many other stakeholders (e.g., legal staff, quality auditors, manufacturing or sales types) who “are often at the periphery of an IXP project Yet they may play important roles on the project” [Ker05].*
- In IXP, the community members and their roles should be explicitly defined and mechanisms for communication and coordination between community members should be established.

3. Project chartering.

- The IXP team assesses the project itself to determine

whether an appropriate business justification for the project exists and whether the project will further the overall goals and objectives of the organization.

- Chartering also examines the context of the project to determine how it complements, extends, or replaces existing systems or processes.

4. Test-driven management:

- An IXP project requires measurable criteria for assessing the state of the project and the progress that has been made to date.

- Test-driven management establishes a series of measurable “destinations” [Ker05] and then defines mechanisms for determining whether or not these destinations have been reached.

5. Retrospectives:

- An IXP team conducts a specialized technical review after a software increment is delivered. Called a *retrospective*,
- The review examines “issues, events, and lessons-learned” [Ker05] across a software increment and/or the entire software release. With an intent to improve the IXP process.

6. Continuous learning:

- Because learning is a vital part of continuous process improvement, members of the XP team are encouraged (and possibly, incented) to learn new methods and techniques that can lead to a higher quality product.

- In addition to the six new practices discussed, IXP modifies a number of existing XP practices.
- *Story-driven development (SDD)* insists that stories for acceptance tests be written before a single line of code is generated.
- *Domain-driven design (DDD)* is an improvement on the “system metaphor” concept used in XP.
- *Pairing* extends the XP pair programming concept to include managers and other stakeholders.
- IXP makes **smaller modifications** to other XP practices and redefines certain roles and responsibilities to make them more amenable to significant projects for large organizations.

The XP Debate

All new process models and methods spur worthwhile discussion and in some instances heated debate. Among the issues that continue to trouble some critics of XP are:

- **Requirements volatility:** customer is an active member of XP team, changes to requirements are requested informally and frequently.
- **Conflicting customer needs:** different customers' needs need to be assimilated by the team. Job beyond their scope of authority.
- **Requirements are expressed informally:** Use stories and acceptance tests are the only explicit manifestation of requirements. Formal models may avoid inconsistencies and errors before the system is built. Proponents said changing nature makes such models obsolete as soon as they are developed.
- **Lack of formal design:** XP deemphasizes the need for architectural design. Critics argue that Complex systems need overall structure to exhibit quality and maintainability. Proponents said incremental nature limits complexity as simplicity is a core value.

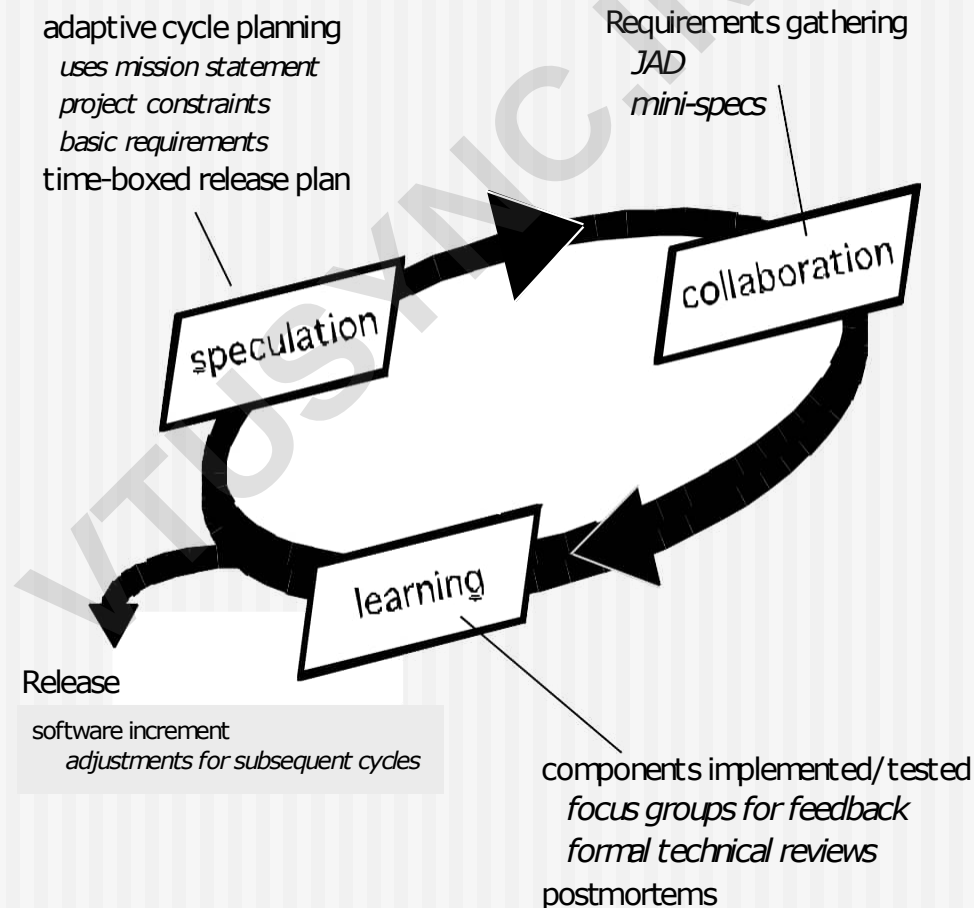
3.5 OTHER AGILE PROCESS MODELS

- As noted in the last section, the most widely used of all agile process models is Extreme Programming (XP).
- But many other agile process models have been proposed and are in use across the industry. Among the most common are:
 1. Adaptive Software Development (ASD)
 2. Scrum
 3. Dynamic Systems Development Method (DSDM)
 4. Crystal
 5. Feature Drive Development (FDD)
 6. Lean Software Development (LSD)
 7. Agile Modeling (AM)
 8. Agile Unified Process (AUP)

Adaptive Software Development (ASD)

- Originally proposed by Jim Highsmith (2000) focusing on human collaboration and team self-organization as a technique to build complex software and system.
- He defines an ASD “life cycle” (Figure 3.3) that incorporates three phases
 1. Speculation
 2. Collaboration
 3. learning.

Adaptive Software Development



Three Phases of ASD

1. Speculation:

- project is initiated and adaptive cycle planning is conducted.
- Adaptive cycle planning uses project initiation information- the customer's mission statement, project constraints (e.g. delivery date), and basic requirements to define the set of release cycles (increments) that will be required for the project.
- Based on the information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality in which an ASD team is working.

Three Phases of ASD

2. Collaborations

- Are used to multiply their talent and creative output beyond absolute number ($1+1>2$).
- It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking.
- It is a matter of trust. People working together must trust one another to
 1. criticize without animosity
 2. assist without resentments
 3. work as hard as or harder than they do
 4. have the skill set to contribute to the work at hand
 5. communicate problems or concerns in a way that leads to effective action.

Three Phases of ASD

3. Learning:

- As members of ASD team begin to develop the components, the emphasis is on “**learning**”.
- Highsmith argues that software developers often overestimate their own understanding of the technology, the process, and the project and that learning will help them to improve their level of real understanding.
- ASD teams learn in three ways:
 1. focus groups,
 2. technical reviews and
 3. project postmortems.

Scrum

- Scrum is an agile software development method that was conceived by **Jeff Sutherland** and his development team in the early 1990s. In recent years, further development on the Scrum methods has been performed by **Schwaber and Beedle** [Sch01a].
- Scrum process incorporates the following framework activities: **requirements, analysis, design, evolution, and delivery.**
- Within each framework activity, work tasks occur within a process pattern called a *sprint*.
- *The work conducted within a sprint* is adapted to the problem at hand and is defined and often modified in real time by the Scrum team.
- *the number of sprints required for each framework activity will vary depending on product complexity and size)*

•Scrum emphasizes the use of a set of software process patterns (that have proven effective for projects with tight timelines, changing requirements, and business Criticality) Each of these process patterns defines a set of development actions:

1. *Backlog*—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time. The product manager assesses the backlog and updates priorities as required.
2. *Sprints*—consist of work units that are required to achieve a requirement defined in the backlog that must be fit into a predefined time-box¹⁴ (typically 30 days). Changes (e.g., backlog work items) are not introduced during the sprint

3. Scrum meetings— *short (typically 15 minutes) meetings held daily by the Scrum team.* Three key questions are asked and answered by all team members

- i. What did you do since the last team meeting?
- ii. What obstacles are you encountering?
- iii. What do you plan to accomplish by the next team meeting?

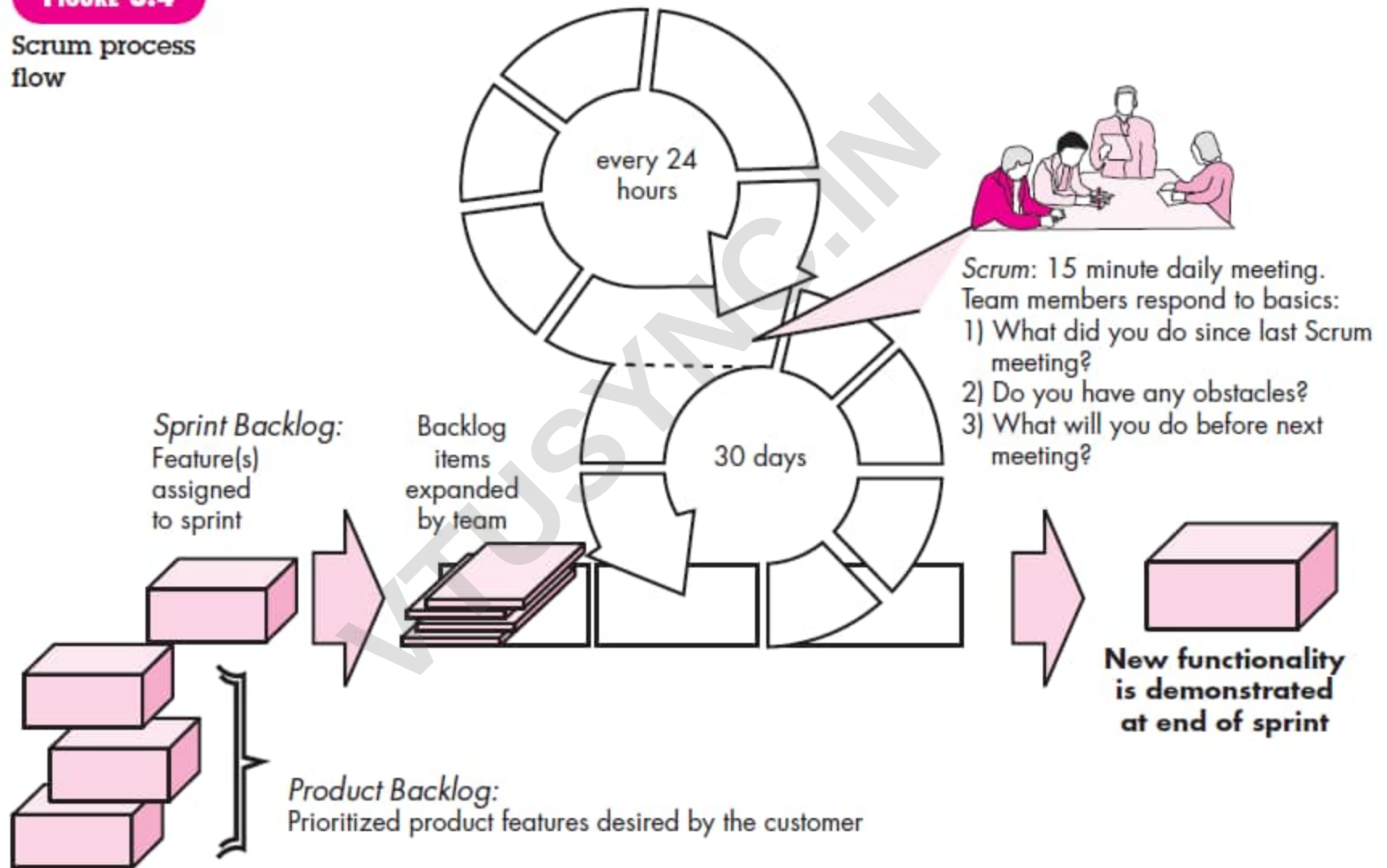
- *Scrum master, leads the meeting and assesses the responses.*
- The Scrum meeting helps the team to uncover potential problems as early as possible, lead to “knowledge socialization & promote a self-organizing team structure.

4. Demos

- *deliver the software increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer.*
- demo may not contain all planned functionality, but rather those functions that can be delivered within the time-box that was established.

FIGURE 3.4

Scrum process
flow



Dynamic Systems Development Method

- It is an agile software development approach that *“provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment”*
- The DSDM philosophy is borrowed from a modified version of the Pareto principle(80% Rule) 80 percent of an application can be delivered in 20 percent of the time it would take to deliver the complete (100 percent) application.
- The **DSDM Consortium** (www.dsdm.org) has defined an agile process model, called the *DSDM life cycle that defines three different iterative cycles*, preceded by two additional life cycle activities:

1. Feasibility study

- *establishes the basic business requirements and constraints associated with the application to be built and then assesses whether the application is a viable candidate for the DSDM process.*

2. Business study

- establishes the functional and information requirements that will allow the application to provide business value;
- defines the basic application architecture and identifies the maintainability requirements for the application.

3. Functional model iteration

- produces a set of incremental prototypes that demonstrate functionality for the customer.
- The intent during this iterative cycle is to gather additional requirements by eliciting feedback from users as they exercise the prototype.

4. Design and build iteration

- revisits prototypes built during functional model iteration to ensure that each has been engineered in a manner to provide operational business value for end users.
- In some cases, *functional model iteration and design and build iteration occur concurrently.*

5. *Implementation*—

- *places the latest software increment* into the operational environment. It should be noted that
 - I. the increment may not be 100 percent complete or
 - II. changes may be requested as the increment is put into place.

In either case, DSDM development work continues by returning to the functional model iteration activity.

- DSDM can be combined with XP (Section 3.4) to provide a combination approach that defines a solid process model (the DSDM life cycle) with the nuts and bolts practices (XP) that are required to build software increments.
- In addition, the ASD concepts of collaboration and self-organizing teams can be adapted to a combined process model.

Crystal

- **Alistair Cockburn and Jim Highsmith** created the *Crystal family of agile methods*
- *Intent is to achieve a software development approach that puts a premium on “**maneuverability**”*
- Cockburn characterizes as “a resource limited, cooperative game of invention and communication, with a primary goal of delivering useful, working software and a secondary goal of setting up for the next game”
- To achieve maneuverability, Cockburn and Highsmith have defined a set of methodologies, each with core elements that are common to all, and roles, process patterns, work products, and practice that are unique to each.
- The Crystal family is actually a set of example agile processes that have been proven effective for different types of projects.
- The intent is to allow agile teams to select the member of the crystal family that is most appropriate for their project and environment

Feature Driven Development

- Originally proposed by Peter Coad et al as a object-oriented software engineering process model.
- Stephen Palmer and John Felsing [Pal02] have extended and improved Coad's work, describing an adaptive, agile process that can be applied to moderately sized and larger software projects.
- **FDD adopts a philosophy that**
 - (1) emphasizes collaboration among people on an FDD team
 - (2) manages problem and project complexity using feature-based decomposition followed by the integration of software increments, and
 - (3) communication of technical detail using verbal, graphical, and text-based means.
- **FDD emphasizes software quality assurance activities by**
 - 1. encouraging an incremental development strategy
 - 2. the use of design and code inspections
 - 3. the application of software quality assurance audits, the collection of metrics, and the use of patterns

Feature Driven Development

- In the context of FDD, a *feature* “*is a client-valued function that can be implemented in two weeks or less*”
- The emphasis on the definition of features provides the following benefits:
 - i. Because features are small blocks of deliverable functionality, users can describe them more easily; understand how they relate to one another more readily; and better review them for ambiguity, error, or omissions.
 - ii. Features can be organized into a hierarchical business-related grouping.
 - iii. Since a feature is the FDD deliverable software increment, the team develops operational features every two weeks.
 - iv. Because features are small, their design and code representations are easier to inspect effectively.
 - v. Project planning, scheduling, and tracking are driven by the feature Hierarchy,

Feature Driven Development

•Coad and his colleagues [Coa99] suggest the following template for defining a feature:

<action> the <result> <by| for |of| to> a(n) <object>

Examples of features for an e-commerce application might be:

1. *Add the product to shopping cart*
2. *Display the technical-specifications of the product*
3. *Store the shipping-information for the customer*

•A feature set groups related features into business-related categories and is defined as:

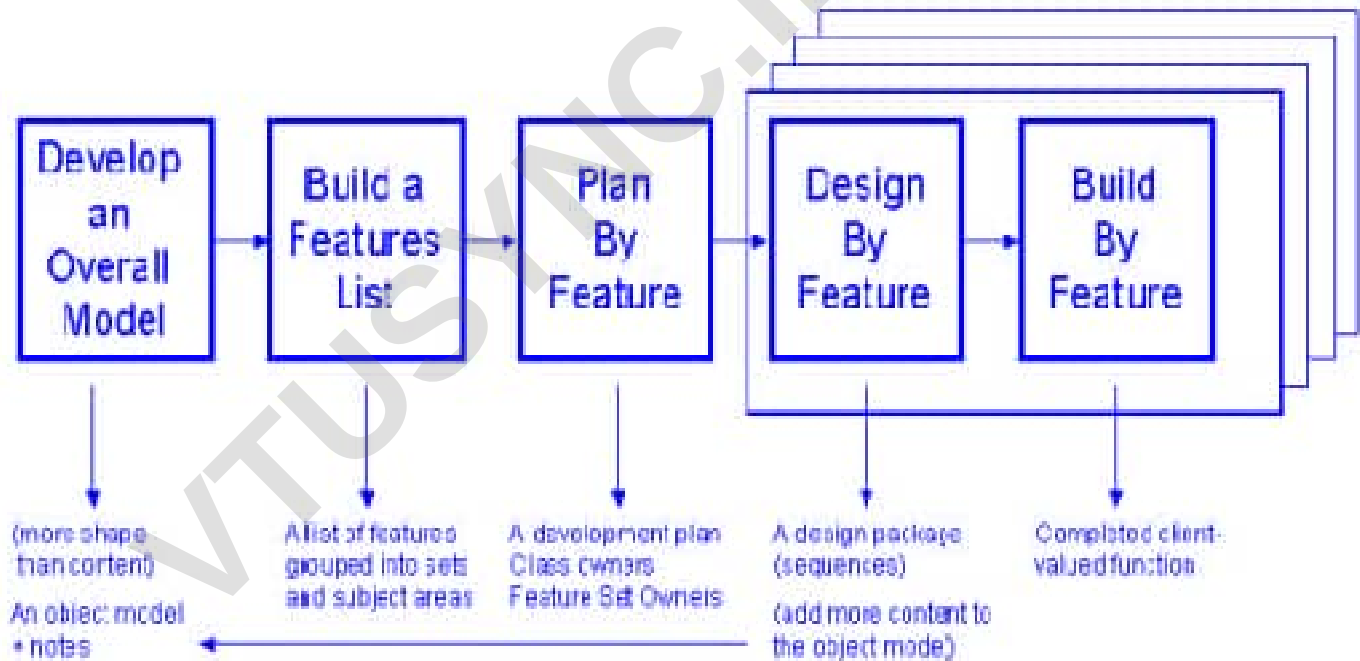
<action><-ing> a(n) <object>

For example: *Making a product sale*

is a feature set that would encompass the features

noted earlier and others.

Feature Driven Development



Reprinted with permission of Peter Coad

Feature Driven Development

- FDD provides greater emphasis on project management guidelines and techniques than many other agile methods.
- As projects grow in size and complexity, ad hoc project management is often inadequate.
- It is essential for developers, their managers, and other stakeholders to understand project status—what accomplishments have been made and problems have been encountered.
- If deadline pressure is significant, it is critical to determine if software increments (features) are properly scheduled.
- To accomplish this, FDD defines six milestones during the design and implementation of a feature: “**design walkthrough, design, design inspection, code, code inspection, promote to build**”.

Lean Software Development

- *Lean Software Development (LSD) has adapted the principles of lean manufacturing to the world of software engineering.*
- *The lean principles that inspire the LSD process can be summarized as **eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, and optimize the whole.***
- *Each of these principles can be adapted to the software process.*
- *For example, **eliminate waste** within the context of an agile software project can be interpreted to mean [Das05]:*
 - (1) adding no extraneous features or functions
 - (2) assessing the cost and schedule impact of any newly requested requirement,
 - (3) removing any superfluous process steps
 - (4) establishing mechanisms to improve the way team members find information
 - (5) ensuring the testing finds as many errors as possible
 - (6) reducing the time required to request and get a decision that affects the software or the process that is applied to create it,
 - (7) streamlining the manner in which information is transmitted to all stakeholders involved in the process.

Agile Modeling

- There are many situations in which software engineers must build large, business critical systems.
- The scope and complexity of such systems must be modeled so that
 - (1) all constituencies can better understand what needs to be accomplished,
 - (2) The problem can be partitioned effectively among the people who must solve it, and
 - (3) quality can be assessed as the system is being engineered and built.
- Over the past 30 years, a wide variety of software engineering modeling methods and notation have been proposed for analysis and design (Having Merits and Demerits)
- Yet analysis and design modeling have substantial benefit for large projects—if for no other reason than to make these projects intellectually manageable.
- Is there an agile approach to software engineering modeling that ?

Agile Modeling

- At “The Official Agile Modeling Site,” Scott Ambler [Amb02a] describes *agile modeling* (AM) in the following manner:

Agile Modeling (AM) is a practice-based methodology for effective modeling and documentation of software-based systems. Simply put,. Agile Modeling (AM) is a collection of values, principles, and practices for modeling software that can be applied on a software development project in an effective and light-weight manner Agile models are more effective than traditional models because they are just barely good, they don't have to be perfect.

- Although AM suggests a wide array of “core” and “supplementary” modeling principles, those that make AM unique are

1. *Model with a purpose:*
2. *Use multiple models:*
3. *Travel light:*
4. *Content is more important than representation:*
5. *Know the models and tools you use to create them:*
6. *Adapt locally:*

Model with a purpose: A developer who uses AM should have a specific goal in mind before creating the model. Once the goal for the model is identified, the type of notation to be used and level of detail required will be more obvious.

Use multiple models: AM suggests that to provide needed insight, each model should present a different aspect of the system and only those models that provide value to their intended audience should be used.

Travel light: As software engineering work proceeds, keep only those models that will provide long-term value and jettison the rest. Every work product that is kept must be maintained as changes occur. This represents work that slows the team down.

Content is more important than representation: Modeling should impart information to its intended audience. A syntactically perfect model that imparts little useful content is not as valuable as a model with flawed notation that nevertheless provides valuable content for its audience.

Know the models and tools you use to create them: Understand the strengths and weaknesses of each model and the tools that are used to create it.

Adapt locally: The modeling approach should be adapted to the needs of the agile team.

Agile Unified Process (AUP)

- The *Agile Unified Process (AUP)* adopts a “serial in the large” and “iterative in the small” philosophy for building computer-based systems.
- By adopting the classic UP phased activities—*inception, elaboration, construction, and transition*—AUP provides a serial overlay
- Each AUP iteration addresses the following activities

1. **Modeling:** UML representations of the business and problem domains are created. However, to stay agile, these models should be “just barely good enough” [Amb06] to allow the team to proceed.
2. **Implementation:** Models are translated into source code.
3. **Testing:** the team designs and executes a series of tests to uncover errors and ensure that the source code meets its requirements.
4. **Deployment:** focuses on the delivery of a software increment and the acquisition of feedback from end users.
5. **Configuration and project management:** Configuration addresses change management, risk management, and the control of any persistent work product that are produced by the team.
Project management tracks and controls the progress of the team and coordinates team activities.
6. **Environment management:** coordinates a process infrastructure that includes standards, tools, and other support technology available to the team.

3.6 A TOOL SET FOR THE AGILE PROCESS

- Some proponents of the agile philosophy argue that automated software tools (e.g., design tools) should be viewed as a minor supplement to the team's activities, and not at all pivotal to the success of the team.
- However, Alistair Cockburn [Coc04] suggests that tools can have a benefit and that *“agile teams stress using tools that permit the rapid flow of understanding. Some of those tools are social, starting even at the hiring stage. Some tools are technological, helping distributed teams simulate being physically present. Many tools are physical, allowing people to manipulate them in workshops.”*
- Acquiring the right people (hiring), team collaboration, stakeholder communication, and indirect management are key elements in virtually all agile process models,
 1. a hiring “tool
 2. Collaborative and communication “tools”
 3. Active communication is achieved via the team dynamics (e.g., pair programming),
 4. passive communication is achieved by “information radiators
 5. Project management tools deemphasize the Gantt chart and replace it with earned value charts or “graphs of tests created versus passed

3.6 A TOOL SET FOR THE AGILE PROCESS

- Other agile tools are used to optimize the environment in which the agile team works(e.g., more efficient meeting areas)
- improve the team culture by nurturing social interactions(e.g., collocated teams), physical devices (e.g., electronic whiteboards), and process enhancement (e.g., pair programming or time-boxing)”
- Are any of these things really tools? They are, if they facilitate the work performed by an agile team member and enhance the quality of the end product.

Principles that guide Practice

- People who create computer software practice the art or craft or discipline that is software engineering.
- But what is software engineering “practice”

In a generic sense, *practice is a collection of concepts, principles, methods, and tools that a software engineer calls upon on a daily basis.*

- Practice allows managers to manage software projects and software engineers to build computer programs.
- Practice populates a software process model with the necessary technical and management how-to's to get the job done.
- Practice transforms a haphazard unfocused approach into something that is more organized, more effective, and more likely to achieve success.



4.1 Software Engineering Knowledge

- You often hear people say that software development knowledge has a 3-year half-life: half of what you need to know today will be obsolete within 3 years. In the domain of technology-related knowledge, that's probably about right. But there is another kind of software development knowledge—a kind that I think of as "software engineering principles"—that does not have a three-year half-life. These software engineering principles are likely to serve a professional programmer throughout his or her career.

Steve McConnell

4.2 Core Principles

- Software engineering is guided by a collection of core principles that help in the application of a meaningful software process and the execution of effective software engineering methods.
- At the process level, core principles establish a philosophical foundation that guides a software team as it performs framework and umbrella activities, navigates the process flow, and produces a set of software engineering work products.
- At the level of practice, core principles establish a collection of values and rules that serve as a guide as you analyze a problem, design a solution, implement and test the solution, and ultimately deploy the software in the user community.



4.2.1 Principles that Guide Process

- **Principle #1. *Be agile.*** Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach.
- **Principle #2. *Focus on quality at every step.*** The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.
- **Principle #3. *Be ready to adapt.*** Process is not a religious experience and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.
- **Principle #4. *Build an effective team.*** Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.



4.2.1 Principles that Guide Process

- **Principle #5. Establish mechanisms for communication and coordination.** Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful end product.
- **Principle #6. Manage change.** The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved and implemented.
- **Principle #7. Assess risk.** Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans.
- **Principle #8. Create work products that provide value for others.** Create only those work products that provide value for other process activities, actions or tasks.



4.2.2 Principles that Guide Practice

- **Principle #1. *Divide and conquer.*** Stated in a more technical manner, analysis and design should always emphasize *separation of concerns* (SoC).
- **Principle #2. *Understand the use of abstraction.*** At its core, an abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase.
- **Principle #3. *Strive for consistency.*** A familiar context makes software easier to use.
- **Principle #4. *Focus on the transfer of information.*** Pay special attention to the analysis, design, construction, and testing of interfaces.



4.2.2 Principles that Guide Practice

- **Principle #5. Build software that exhibits effective modularity.** Separation of concerns (Principle #1) establishes a philosophy for software. *Modularity* provides a mechanism for realizing the philosophy.
- **Principle #6. Look for patterns.** Brad Appleton [App00] suggests that: “The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development.”
- **Principle #7. When possible, represent the problem and its solution from a number of different perspectives.**
- **Principle #8. Remember that someone will maintain the software.**



4.3.1

Communication Principles

- **Principle #1. *Listen.*** Try to focus on the speaker's words, rather than formulating your response to those words.
- **Principle # 2. *Prepare before you communicate.*** Spend the time to understand the problem before you meet with others.
- **Principle # 3. *Someone should facilitate the activity.*** Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction; (2) to mediate any conflict that does occur, and (3) to ensure that other principles are followed.
- **Principle #4. *Face-to-face communication is best.*** But it usually works better when some other representation of the relevant information is present.



4.3.1

Communication Principles

- **Principle # 5. *Take notes and document decisions.*** Someone participating in the communication should serve as a “recorder” and write down all important points and decisions.
- **Principle # 6. *Strive for collaboration.*** Collaboration and consensus occur when the collective knowledge of members of the team is combined ...
- **Principle # 7. *Stay focused, modularize your discussion.*** The more people involved in any communication, the more likely that discussion will bounce from one topic to the next.
- **Principle # 8. *If something is unclear, draw a picture.***
- **Principle # 9. *(a) Once you agree to something, move on; (b) If you can't agree to something, move on; (c) If a feature or function is unclear and cannot be clarified at the moment, move on.***
- **Principle # 10. *Negotiation is not a contest or a game. It works best when both parties win.***



4.3.2

Planning Principles

- **Principle #1. *Understand the scope of the project.*** It's impossible to use a roadmap if you don't know where you're going. Scope provides the software team with a destination.
- **Principle #2. *Involve the customer in the planning activity.*** The customer defines priorities and establishes project constraints.
- **Principle #3. *Recognize that planning is iterative.*** A project plan is never engraved in stone. As work begins, it very likely that things will change.
- **Principle #4. *Estimate based on what you know.*** The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done.



4.3.2

Planning Principles

- **Principle #5. Consider risk as you define the plan.** If you have identified risks that have high impact and high probability, contingency planning is necessary.
- **Principle #6. Be realistic.** People don't work 100 percent of every day.
- **Principle #7. Adjust granularity as you define the plan.** Granularity refers to the level of detail that is introduced as a project plan is developed.
- **Principle #8. Define how you intend to ensure quality.** The plan should identify how the software team intends to ensure quality.
- **Principle #9. Describe how you intend to accommodate change.** Even the best planning can be obviated by uncontrolled change.
- **Principle #10. Track the plan frequently and make adjustments as required.** Software projects fall behind schedule one day at a time.



4.3.3

Modeling Principles

- ❑ In software engineering work, two classes of models can be created:
 - *Requirements models (also called analysis models)* represent the customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain.
 - *Design models* represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail.



Agile Modeling Principles

- **Principle #1.** *The primary goal of the software team is to build software, not create models.*
- **Principle #2.** *Travel light—don't create more models than you need.*
- **Principle #3.** *Strive to produce the simplest model that will describe the problem or the software.*
- **Principle #4.** *Build models in a way that makes them amenable to change.*
- **Principle #5.** *Be able to state an explicit purpose for each model that is created.*
- **Principle #6.** *Adapt the models you develop to the system at hand.*
- **Principle #7.** *Try to build useful models, but forget about building perfect models.*
- **Principle #8.** *Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary.*
- **Principle #9.** *If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned.*
- **Principle #10.** *Get feedback as soon as you can.*



Requirements Modeling Principles

- **Principle #1.** *The information domain of a problem must be represented and understood.*
- **Principle #2.** *The functions that the software performs must be defined.*
- **Principle #3.** *The behavior of the software (as a consequence of external events) must be represented.*
- **Principle #4.** *The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion.*
- **Principle #5.** *The analysis task should move from essential information toward implementation detail.*



Design Modeling Principles

- **Principle #1.** *Design should be traceable to the requirements model.*
- **Principle #2.** *Always consider the architecture of the system to be built.*
- **Principle #3.** *Design of data is as important as design of processing functions.*
- **Principle #5.** User interface design should be tuned to the needs of the end-user. However, in every case, it should stress ease of use.
- **Principle #6.** Component-level design should be functionally independent.
- **Principle #7.** Components should be loosely coupled to one another and to the external environment.
- **Principle #8.** Design representations (models) should be easily understandable.
- **Principle #9.** The design should be developed iteratively. With each iteration, the designer should strive for greater simplicity.
- **Principle #4.** Interfaces (both internal and external) must be designed with care



Construction Principles

- The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end-user.
- **Coding principles and concepts** are closely aligned programming style, programming languages, and programming methods.
- **Testing principles and concepts** lead to the design of tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.



Preparation Principles

❑ ***Before you write one line of code, be sure you:***

- Understand of the problem you're trying to solve.
- Understand basic design principles and concepts.
- Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
- Select a programming environment that provides tools that will make your work easier.
- Create a set of unit tests that will be applied once the component you code is completed.



Coding Principles

❑ ***As you begin writing code, be sure you:***

- Constrain your algorithms by following structured programming [Boh00] practice.
- Consider the use of pair programming
- Select data structures that will meet the needs of the design.
- Understand the software architecture and create interfaces that are consistent with it.
- Keep conditional logic as simple as possible.
- Create nested loops in a way that makes them easily testable.
- Select meaningful variable names and follow other local coding standards.
- Write code that is self-documenting.
- Create a visual layout (e.g., indentation and blank lines) that aids understanding.



Validation Principles

❑ ***After you've completed your first coding pass, be sure you:***

- Conduct a code walkthrough when appropriate.
- Perform unit tests and correct errors you've uncovered.
- Refactor the code.



Testing Principles

- ❑ Al Davis [Dav95] suggests the following:
 - **Principle #1.** *All tests should be traceable to customer requirements.*
 - **Principle #2.** *Tests should be planned long before testing begins.*
 - **Principle #3.** *The Pareto principle applies to software testing.*
 - **Principle #4.** *Testing should begin “in the small” and progress toward testing “in the large.”*
 - **Principle #5.** *Exhaustive testing is not possible.*



Deployment Principles

- **Principle #1. *Customer expectations for the software must be managed.*** Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately.
- **Principle #2. *A complete delivery package should be assembled and tested.***
- **Principle #3. *A support regime must be established before the software is delivered.*** An end-user expects responsiveness and accurate information when a question or problem arises.
- **Principle #4. *Appropriate instructional materials must be provided to end-users.***
- **Principle #5. *Buggy software should be fixed first, delivered later.***